



ЗВІТ

Лабораторна робота № 3

Дисципліна: Базы даних та інформаційні системи

Тема: Проектування та створення бази даних

Виконав

Студент 3 курсу

Групи МІТ-31

Факультету інформаційних технологій

Бабяк Володимир

Мета роботи

Закріплення теоретичних знань та набуття практичних навичок у складанні складних SQL-запитів до реляційної бази даних. Робота передбачає використання операторів SELECT, WHERE, логічних операторів, агрегатних функцій, JOIN, підзапитів, CTE, віконних функцій тощо.

Хід роботи

Завдання 1. Розробити 40 SQL-запитів до створеної у попередній роботі бази даних, що охоплюють наступні аспекти:

а. Логічні оператори та базова вибірка:

```
1  -- Запит 1. Отримати всі поїздки, які були успішно завершені та коштували більше 200
2  SELECT * FROM rides
3  WHERE status = 'completed' AND price > 200;
```

Data Output Messages Notifications

Showing rows: 1 to 4

	ride_id [PK] integer	client_id integer	driver_id integer	ride_date timestamp without time zone	distance_km numeric (5,2)	price numeric (7,2)	status character varying (20)
1	1	1	1	2026-05-01 08:30:00	12.50	250.00	completed
2	4	4	3	2026-05-02 18:45:00	22.40	450.00	completed
3	6	1	3	2026-05-03 22:10:00	15.00	300.00	completed
4	9	4	1	2026-05-05 19:00:00	18.20	380.00	completed

1. Запит повертає всі успішно завершені поїздки, вартість яких перевищує 200 (використання логічного оператора AND).

```
1  -- Запит 2. Знайти водіїв, які їздять на Toyota або Skoda
2  SELECT first_name, last_name, car_brand
3  FROM drivers
4  WHERE car_brand = 'Toyota' OR car_brand = 'Skoda';
```

Data Output Messages Notifications

	first_name character varying (50) 🔒	last_name character varying (50) 🔒	car_brand character varying (50) 🔒
1	Андрій	Мельник	Toyota
2	Володимир	Сидоренко	Skoda

2. Запит показує водіїв, які їздять на автомобілях марок Toyota або Skoda (використання оператора OR).

```

1  -- Запит 3. Знайти клієнтів, чиї номери телефонів не починаються на '+38050'
2  SELECT first_name, last_name, phone_number
3  FROM clients
4  WHERE NOT phone_number LIKE '+38050%';

```

Data Output Messages Notifications

	first_name character varying (50)	last_name character varying (50)	phone_number character varying (20)
1	Марія	Шевченко	+380671234567
2	Іван	Бойко	+380631234567
3	Олена	Ткаченко	+380991234567
4	Дмитро	Кравченко	+380971234567

3. Запит знаходить клієнтів, чиї номери телефонів не починаються на код '+38050' (використання NOT LIKE).

```

1  -- Запит 4. Вибрати поїздки з дистанцією від 5 до 15 км
2  SELECT * FROM rides
3  WHERE distance_km BETWEEN 5 AND 15;

```

Data Output Messages Notifications

	ride_id [PK] integer	client_id integer	driver_id integer	ride_date timestamp without time zone	distance_km numeric (5,2)	price numeric (7,2)	status character varying (20)
1	1	1	1	2026-05-01 08:30:00	12.50	250.00	completed
2	2	2	2	2026-05-01 09:15:00	5.20	120.00	completed
3	3	3	1	2026-05-02 14:00:00	8.00	160.00	completed
4	6	1	3	2026-05-03 22:10:00	15.00	300.00	completed
5	7	2	1	2026-05-04 07:50:00	9.50	190.00	completed
6	10	5	3	2026-05-06 11:15:00	7.70	150.00	completed

Showing rows: 1 to 6

4. Вибірка поїздок, дистанція яких становить від 5 до 15 км (використання BETWEEN).

```

1  -- Запит 5. Знайти всіх клієнтів, у яких ім'я 'Олександр' або 'Марія'
2  SELECT * FROM clients
3  WHERE first_name IN ('Олександр', 'Марія');
4

```

Data Output Messages Notifications

	client_id [PK] integer	first_name character varying (50)	last_name character varying (50)	phone_number character varying (20)
1	1	Олександр	Коваленко	+380501234567
2	2	Марія	Шевченко	+380671234567

5. Пошук клієнтів із конкретними іменами 'Олександр' або 'Марія' (використання IN).

i. Агрегатні функції (COUNT, SUM, AVG, MIN, MAX):

```

1  -- Запит 6. Підрахувати загальну кількість скасованих поїздки
2  SELECT COUNT(*) AS cancelled_rides_count
3  FROM rides
4  WHERE status = 'cancelled';
5

```

Data Output Messages Notifications

	cancelled_rides_count bigint
1	1

6. Підрахунок загальної кількості скасованих поїздки (COUNT).

```
1 -- Запит 7. Знайти загальну суму грошей, зароблену всіма водіями на успішних поїздах
2 SELECT SUM(price) AS total_revenue
3 FROM rides
4 WHERE status = 'completed';
```

Data Output Messages Notifications

	total_revenue numeric
1	2100.00

7. Загальна сума грошей, зароблене на всіх успішних поїздах (SUM).

```
1 -- Запит 8. Обчислити середню відстань усіх поїздок (з округленням до 2 знаків)
2 SELECT ROUND(AVG(distance_km), 2) AS avg_distance
3 FROM rides;
```

Data Output Messages Notifications

	avg_distance numeric
1	10.56

8. Середня дистанція усіх поїздок у базі з округленням до двох знаків (AVG).

Query Query History

```

1  -- Запит 9. Знайти мінімальну та максимальну вартість поїздки
2  SELECT MIN(price) AS min_price, MAX(price) AS max_price
3  FROM rides;|

```

Data Output Messages Notifications

	min_price numeric	max_price numeric
1	85.00	450.00

9. Пошук найдешевшої (MIN) та найдорожчої (MAX) вартості поїздки.

```

1  -- Запит 10. Підрахувати кількість поїздок для кожного статусу
2  SELECT status, COUNT(ride_id) AS ride_count
3  FROM rides
4  GROUP BY status;|

```

Data Output Messages Notifications

	status character varying (20)	ride_count bigint
1	completed	9
2	cancelled	1

10. Підрахунок кількості поїздок, згрупованих за кожним статусом (GROUP BY + COUNT).

ii. Усі типи JOIN (INNER, LEFT, RIGHT, FULL, CROSS, SELF):

Query Query History

```

1  -- Запит 11. INNER JOIN: Отримати імена клієнтів та дати їхніх поїздок
2  SELECT c.first_name, c.last_name, r.ride_date
3  FROM clients c
4  INNER JOIN rides r ON c.client_id = r.client_id;

```

Data Output Messages Notifications

	first_name character varying (50)	last_name character varying (50)	ride_date timestamp without time zone
1	Олександр	Коваленко	2026-05-01 08:30:00
2	Марія	Шевченко	2026-05-01 09:15:00
3	Іван	Бойко	2026-05-02 14:00:00
4	Олена	Ткаченко	2026-05-02 18:45:00
5	Дмитро	Кравченко	2026-05-03 10:20:00
6	Олександр	Коваленко	2026-05-03 22:10:00
7	Марія	Шевченко	2026-05-04 07:50:00
8	Іван	Бойко	2026-05-04 13:30:00
9	Олена	Ткаченко	2026-05-05 19:00:00
10	Дмитро	Кравченко	2026-05-06 11:15:00

11. INNER JOIN: Запит повертає імена клієнтів та дати їхніх поїздок (тільки тих, хто їздив).

```

1  -- Запит 12. LEFT JOIN: Вивести всіх клієнтів і їхні поїздки (навіть якщо поїздок ще не було)
2  SELECT c.first_name, COUNT(r.ride_id) AS total_rides
3  FROM clients c
4  LEFT JOIN rides r ON c.client_id = r.client_id
5  GROUP BY c.first_name;

```

Data Output Messages Notifications

	first_name character varying (50)	total_rides bigint
1	Олена	2
2	Марія	2
3	Дмитро	2
4	Іван	2
5	Олександр	2

Showing rows: 1 to 5

Query		Query History
1	-- Запит 12. LEFT JOIN: Вивести всіх клієнтів і їхні поїздки (навіть якщо поїздки ще не було)	
2	SELECT c.first_name, COUNT(r.ride_id) AS total_rides	
3	FROM clients c	
4	LEFT JOIN rides r ON c.client_id = r.client_id	
5	GROUP BY c.first_name;	

Data Output		Messages	Notifications
+ 📄 ▼ 📋 ▼ 🗑️ 🗑️ 📥 ⬇️ 📈 SQL Showing rows: 1 to 5			
	first_name	total_rides	
	character varying (50)	bigint	
1	Олена	2	
2	Марія	2	
3	Дмитро	2	
4	Іван	2	
5	Олександр	2	

12. LEFT JOIN: Запит показує всіх клієнтів і кількість їхніх поїздки, навіть якщо поїздки ще не було (буде 0).

1	-- Запит 13. RIGHT JOIN: Вивести всіх водіїв і суму їхніх заробітків (навіть якщо вони ще не їздили)	
2	SELECT d.first_name, SUM(r.price) AS earned_money	
3	FROM rides r	
4	RIGHT JOIN drivers d ON r.driver_id = d.driver_id	
5	GROUP BY d.first_name;	

Data Output		Messages	Notifications
+ 📄 ▼ 📋 ▼ 🗑️ 🗑️ 📥 ⬇️ 📈 SQL Showing rows: 1 to 3			
	first_name	earned_money	
	character varying (50)	numeric	
1	Сергій	305.00	
2	Володимир	900.00	
3	Андрій	980.00	

13. RIGHT JOIN: Запит повертає всіх водіїв і суму їхніх заробітків, навіть якщо вони ще не виконали жодного замовлення.

Query
Query History

```

1  -- Запит 14. FULL OUTER JOIN: Об'єднати дані клієнтів та поїздки повністю
2  SELECT c.first_name, r.ride_date, r.status
3  FROM clients c
4  FULL OUTER JOIN rides r ON c.client_id = r.client_id;

```

Data Output
Messages
Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

S

	first_name character varying (50) 🔒	ride_date timestamp without time zone 🔒	status character varying (20) 🔒
1	Олександр	2026-05-01 08:30:00	completed
2	Марія	2026-05-01 09:15:00	completed
3	Іван	2026-05-02 14:00:00	completed
4	Олена	2026-05-02 18:45:00	completed
5	Дмитро	2026-05-03 10:20:00	cancelled
6	Олександр	2026-05-03 22:10:00	completed
7	Марія	2026-05-04 07:50:00	completed
8	Іван	2026-05-04 13:30:00	completed
9	Олена	2026-05-05 19:00:00	completed
10	Дмитро	2026-05-06 11:15:00	completed

14. FULL OUTER JOIN: Повне об'єднання даних клієнтів та поїздки.

```
1  -- Запит 15. CROSS JOIN: Створити всі можливі комбінації пар "клієнт - водій"
2  SELECT c.first_name AS client, d.first_name AS driver
3  FROM clients c
4  CROSS JOIN drivers d;
```

s Data Output Messages Notifications

SQL

	client character varying (50) 🔒	driver character varying (50) 🔒
1	Олександр	Андрій
2	Олександр	Сергій
3	Олександр	Володимир
4	Марія	Андрій
5	Марія	Сергій
6	Марія	Володимир
7	Іван	Андрій
8	Іван	Сергій
9	Іван	Володимир
10	Олена	Андрій
11	Олена	Сергій
12	Олена	Володимир
13	Дмитро	Андрій
14	Дмитро	Сергій
15	Дмитро	Володимир

15. CROSS JOIN: Запит створює всі можливі комбінації пар "клієнт — водій" (декартовий добуток).

```

1  -- Запит 16. SELF JOIN: Знайти водіїв, які їздять на марках авто з однаковою першою літерою (наприклад, Toyota)
2  SELECT d1.first_name AS driver1, d2.first_name AS driver2, d1.car_brand
3  FROM drivers d1
4  JOIN drivers d2 ON d1.car_brand = d2.car_brand AND d1.driver_id != d2.driver_id;

```

	driver1 character varying (50)	driver2 character varying (50)	car_brand character varying (50)
1	Андрій	Олег	Toyota
2	Олег	Андрій	Toyota

16. SELF JOIN: Запит знаходить пари різних водіїв, які працюють на автомобілях однакової марки.

```

1  -- Запит 17. Складний JOIN (3 таблиці): Детальна інформація про поїздку (Клієнт + Водій + Поїздка)
2  SELECT c.first_name AS client, d.first_name AS driver, r.distance_km, r.price
3  FROM rides r
4  JOIN clients c ON r.client_id = c.client_id
5  JOIN drivers d ON r.driver_id = d.driver_id;

```

	client character varying (50)	driver character varying (50)	distance_km numeric (5,2)	price numeric (7,2)
1	Олександр	Андрій	12.50	250.00
2	Марія	Сергій	5.20	120.00
3	Іван	Андрій	8.00	160.00
4	Олена	Володимир	22.40	450.00
5	Дмитро	Сергій	3.10	85.00
6	Олександр	Володимир	15.00	300.00
7	Марія	Андрій	9.50	190.00
8	Іван	Сергій	4.00	100.00
9	Олена	Андрій	18.20	380.00
10	Дмитро	Володимир	7.70	150.00

17. Складний JOIN: Об'єднання трьох таблиць для виведення детальної інформації про поїздку (Ім'я клієнта, Ім'я водія, Дистанція, Вартість).

b. Складні запити:

i. Підзапити (Subqueries) у WHERE, IN, NOT EXISTS, EXISTS:

Query Query History

```
1 -- Запит 18. Підзапит у WHERE (IN): Водії, які виконали поїздки на дистанцію більше 15 км
2 SELECT first_name, last_name, car_brand
3 FROM drivers
4 WHERE driver_id IN (SELECT driver_id FROM rides WHERE distance_km > 15);
```

Data Output Messages Notifications

Showing rows: 1 to 2

	first_name character varying (50)	last_name character varying (50)	car_brand character varying (50)
1	Володимир	Сидоренко	Skoda
2	Андрій	Мельник	Toyota

18. Підзапит з IN: Знаходить водіїв, які виконували довгі поїздки (більше 15 км).

Query Query History

```
1 -- Запит 19. Підзапит (NOT IN): Клієнти, які ніколи не скасовували поїздки
2 SELECT first_name, last_name
3 FROM clients
4 WHERE client_id NOT IN (SELECT client_id FROM rides WHERE status = 'cancelled');
```

Data Output Messages Notifications

Showing rows: 1 to 4

	first_name character varying (50)	last_name character varying (50)
1	Олександр	Коваленко
2	Марія	Шевченко
3	Іван	Бойко
4	Олена	Ткаченко

19. Підзапит з NOT IN: Повертає клієнтів, які ніколи не скасовували свої поїздки.

```
1  -- Запит 20. EXISTS: Знайти клієнтів, які здійснили хоча б одну поїздку дорожче 300
2  SELECT first_name, last_name
3  FROM clients c
4  WHERE EXISTS (
5      SELECT 1 FROM rides r WHERE r.client_id = c.client_id AND r.price > 300
6  );
```

Data Output Messages Notifications

	first_name character varying (50)	last_name character varying (50)
1	Олена	Ткаченко

20. Підзапит з EXISTS: Знаходить клієнтів, які здійснили хоча б одну поїздку вартістю понад 300.

```
1  -- Запит 21. NOT EXISTS: Знайти водіїв, які не виконали жодної поїздки
2  SELECT first_name, last_name
3  FROM drivers d
4  WHERE NOT EXISTS (
5      SELECT 1 FROM rides r WHERE r.driver_id = d.driver_id
6  );
```

Data Output Messages Notifications

	first_name character varying (50)	last_name character varying (50)
1	Олег	Іванов

21. Підзапит з NOT EXISTS: Знаходить водіїв, які ще не виконали жодної поїздки в системі.

Query

Query History

1

-- Запит 22. Підзапит у SELECT: Вивести клієнта і відсоток його витрат від загальної суми всіх поїздок

2

SELECT first_name,

3

(SELECT SUM(price) FROM rides WHERE client_id = c.client_id) AS personal_spend,

4

(SELECT SUM(price) FROM rides) AS total_all_spend

5

FROM clients c;

Data Output

Messages

Notifications

≡

+

📄

▼

📄

▼

🗑

🗑

📦

⬇

📈

⬇

SQL

Showing rows: 1 to 5

✎

🖨

	first_name	personal_spend	total_all_spend
	character varying (50)	numeric	numeric
1	Олександр	550.00	2185.00
2	Марія	310.00	2185.00
3	Іван	260.00	2185.00
4	Олена	830.00	2185.00
5	Дмитро	235.00	2185.00

22. Підзапит у SELECT: Виводить ім'я клієнта, його персональні витрати та загальну суму всіх витрат усіх клієнтів поруч для порівняння.

1

-- Запит 23. Підзапит у FROM: Знайти середній чек серед клієнтів, які витратили більше 200

2

SELECT ROUND(AVG(total_spent), 2) AS avg_top_spenders

3

FROM (

4

SELECT client_id, SUM(price) AS total_spent

5

FROM rides

6

GROUP BY client_id

7

HAVING SUM(price) > 200

8

) AS top_clients;

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

📦

⬇️

📈

SQL

Showing rows: 1 to 1

	avg_top_spenders
	numeric
1	437.00

23. Підзапит у FROM: Знаходить середній чек серед так званих "топ-клієнтів" (тих, хто витратив більше 200).

```

1  -- Запит 24. Вкладений підзапит (Глибокий): Клієнти, які їздили з водієм на 'Toyota'
2  SELECT first_name, last_name
3  FROM clients
4  WHERE client_id IN (
5      SELECT client_id FROM rides WHERE driver_id IN (
6          SELECT driver_id FROM drivers WHERE car_brand = 'Toyota'
7      )
8  );

```

Data Output Messages Notifications

	first_name character varying (50)	last_name character varying (50)
1	Олександр	Коваленко
2	Іван	Бойко
3	Олена	Ткаченко
4	Марія	Шевченко

24. Вкладений підзапит (багатоступеневий): Знаходить клієнтів, які їздили з водієм на автомобілі марки 'Toyota'.

ii. Операції над множинами (UNION, INTERSECT, EXCEPT):

```

1  -- Запит 25. UNION: Отримати єдиний список усіх номерів телефонів (клієнтів та водіїв)
2  SELECT phone_number, 'Client' AS role FROM clients
3  UNION
4  SELECT phone_number, 'Driver' AS role FROM drivers;

```

Data Output Messages Notifications

	phone_number character varying (20)	role text
1	+380990001122	Driver
2	+380671234567	Client
3	+380661112233	Driver
4	+380971234567	Client
5	+380933334455	Driver
6	+380501234567	Client
7	+380502223344	Driver
8	+380991234567	Client
9	+380631234567	Client

25. UNION: Запит створює єдиний довідник номерів телефонів (об'єднує телефони клієнтів та водіїв без дублікатів).


```
1  -- Запит 26. UNION ALL: Отримати всі імена з бази (включаючи дублікати)
2  SELECT first_name FROM clients
3  UNION ALL
4  SELECT first_name FROM drivers;
```

Data Output Messages Notifications

	first_name character varying (50) 🔒
1	Олександр
2	Марія
3	Іван
4	Олена
5	Дмитро
6	Андрій
7	Сергій
8	Володимир
9	Олег

26. UNION ALL: Отримання списку абсолютно всіх імен у базі (клієнти + водії), включаючи дублікати.

```
1  -- Запит 27. INTERSECT: Знайти імена, які зустрічаються і серед клієнтів, і серед водіїв
2  SELECT first_name FROM clients
3  INTERSECT
4  SELECT first_name FROM drivers;
```

Data Output Messages Notifications

	first_name character varying (50) 🔒
1	Олександр

Showing rows: 1 to 1

27. INTERSECT: Знаходить імена, які зустрічаються одночасно і серед клієнтів, і серед водіїв (перетин множин).

```

1  -- Запит 28. EXCEPT: Знайти імена клієнтів, яких немає серед імен водіїв
2  SELECT first_name FROM clients
3  EXCEPT
4  SELECT first_name FROM drivers;

```

Data Output Messages Notifications

	first_name character varying (50)
1	Олена
2	Марія
3	Дмитро
4	Іван

28. EXCEPT: Повертає імена клієнтів, яких немає серед списку імен водіїв (різниця множин).

iii. Common Table Expressions (CTE):

```

1  -- Запит 29. Простий CTE: Вивести поїздки тільки успішних статусів
2  WITH CompletedRides AS (
3      SELECT * FROM rides WHERE status = 'completed'
4  )
5  SELECT * FROM CompletedRides WHERE price > 150;

```

Data Output Messages Notifications

	ride_id [PK] integer	client_id integer	driver_id integer	ride_date timestamp without time zone	distance_km numeric (5,2)	price numeric (7,2)	status character varying (20)
1	1	1	1	2026-05-01 08:30:00	12.50	250.00	completed
2	3	3	1	2026-05-02 14:00:00	8.00	160.00	completed
3	4	4	3	2026-05-02 18:45:00	22.40	450.00	completed
4	6	1	3	2026-05-03 22:10:00	15.00	300.00	completed
5	7	2	1	2026-05-04 07:50:00	9.50	190.00	completed
6	9	4	1	2026-05-05 19:00:00	18.20	380.00	completed

29. Простий CTE: Створює тимчасову таблицю успішних поїздок і робить вибірку з неї за ціною > 150.

```

1  -- Запит 30. CTE з агрегацією: Знайти водіїв, які заробили більше середнього
2  WITH DriverEarnings AS (
3      SELECT driver_id, SUM(price) AS total_earned
4      FROM rides
5      GROUP BY driver_id
6  ),
7  AvgEarn AS (
8      SELECT AVG(total_earned) AS avg_e FROM DriverEarnings
9  )
10 SELECT d.first_name, de.total_earned
11 FROM drivers d
12 JOIN DriverEarnings de ON d.driver_id = de.driver_id
13 WHERE de.total_earned > (SELECT avg_e FROM AvgEarn);

```

Data Output Messages Notifications

	first_name character varying (50)	total_earned numeric
1	Андрій	980.00
2	Володимир	900.00

30. CTE з агрегацією: Аналітичний запит, що знаходить водіїв, які заробили більше, ніж середній показник по всій службі таксі.

```

1  -- Запит 31. CTE для підрахунку поїздок клієнта
2  WITH ClientRidesCount AS (
3      SELECT client_id, COUNT(ride_id) AS num_rides
4      FROM rides
5      GROUP BY client_id
6  )
7  SELECT c.first_name, crc.num_rides
8  FROM clients c
9  JOIN ClientRidesCount crc ON c.client_id = crc.client_id
10 ORDER BY crc.num_rides DESC;

```

Data Output Messages Notifications

	first_name character varying (50)	num_rides bigint
1	Олександр	2
2	Марія	2
3	Іван	2
4	Олена	2
5	Дмитро	2

31. СТЕ для підрахунку: Повертає клієнтів та кількість їхніх поїздок, відсортованих за спаданням активності.

```
1  -- Запит 32. СТЕ: Найдорожча поїздка в базі
2  WITH MaxPriceRide AS (
3      SELECT MAX(price) AS max_p FROM rides
4  )
5  SELECT * FROM rides WHERE price = (SELECT max_p FROM MaxPriceRide);
```

Data Output Messages Notifications

Showing rows: 1 to 1

	ride_id [PK] integer	client_id integer	driver_id integer	ride_date timestamp without time zone	distance_km numeric (5,2)	price numeric (7,2)	status character varying (20)
1	4	4	3	2026-05-02 18:45:00	22.40	450.00	completed

32. СТЕ (пошук максимуму): Виводить повну інформацію про найдорожчу поїздку в базі.

```
1  -- Запит 33. Складний СТЕ з JOIN: Деталі скасованих замовлень
2  WITH CancelledDetails AS (
3      SELECT r.ride_id, r.client_id, r.driver_id, r.ride_date
4      FROM rides r WHERE status = 'cancelled'
5  )
6  SELECT cd.ride_id, c.first_name AS client, d.first_name AS driver
7  FROM CancelledDetails cd
8  JOIN clients c ON cd.client_id = c.client_id
9  JOIN drivers d ON cd.driver_id = d.driver_id;
```

Data Output Messages Notifications

	ride_id integer	client character varying (50)	driver character varying (50)
1	5	Дмитро	Сергій

33. Складний СТЕ з JOIN: Формує деталізований звіт про скасовані замовлення з підтягуванням імен клієнтів та водіїв.

```

1  -- Запит 34. Багатоступеневий CTE: Аналіз прибутковості по датах
2  WITH DailyRevenue AS (
3      SELECT DATE(ride_date) AS ride_day, SUM(price) AS daily_total
4      FROM rides
5      WHERE status = 'completed'
6      GROUP BY DATE(ride_date)
7  )
8  SELECT * FROM DailyRevenue ORDER BY daily_total DESC;
9

```

	ride_day date	daily_total numeric
1	2026-05-...	610.00
2	2026-05-...	380.00
3	2026-05-...	370.00
4	2026-05-...	300.00
5	2026-05-...	290.00
6	2026-05-...	150.00

34. Багатоступеневий CTE: Аналіз прибутковості компанії по днях (згруповано за датою).

iv. Віконні функції (Window Functions):

```

1  -- Запит 35. OVER(): Показати вартість поїздки та загальну суму всіх поїздок поруч
2  SELECT ride_id, price,
3         SUM(price) OVER() AS grand_total
4  FROM rides;

```

	ride_id [PK] integer	price numeric (7,2)	grand_total numeric
1	1	250.00	2185.00
2	2	120.00	2185.00
3	3	160.00	2185.00
4	4	450.00	2185.00
5	5	85.00	2185.00
6	6	300.00	2185.00
7	7	190.00	2185.00
8	8	100.00	2185.00
9	9	380.00	2185.00
10	10	150.00	2185.00

35. OVER(): Показує вартість конкретної поїздки та загальну суму всіх поїздок бази у кожному рядку.

```
1  -- Запит 36. PARTITION BY: Середня вартість поїздки для кожного конкретного водія
2  SELECT ride_id, driver_id, price,
3         ROUND(AVG(price) OVER(PARTITION BY driver_id), 2) AS avg_driver_price
4  FROM rides;
```

	ride_id [PK] integer	driver_id integer	price numeric (7,2)	avg_driver_price numeric
1	1	1	250.00	245.00
2	3	1	160.00	245.00
3	7	1	190.00	245.00
4	9	1	380.00	245.00
5	2	2	120.00	101.67
6	8	2	100.00	101.67
7	5	2	85.00	101.67
8	10	3	150.00	300.00
9	4	3	450.00	300.00
10	6	3	300.00	300.00

36. PARTITION BY: Обчислює середню вартість поїздки індивідуально для кожного водія і виводить її поруч із деталями поїздки.

```
1  -- Запит 37. ROW_NUMBER(): Пронумерувати поїздки кожного клієнта за датою (від найпершої)
2  SELECT client_id, ride_id, ride_date,
3         ROW_NUMBER() OVER(PARTITION BY client_id ORDER BY ride_date) AS ride_sequence
4  FROM rides;
```

	client_id integer	ride_id [PK] integer	ride_date timestamp without time zone	ride_sequence bigint
1	1	1	2026-05-01 08:30:00	1
2	1	6	2026-05-03 22:10:00	2
3	2	2	2026-05-01 09:15:00	1
4	2	7	2026-05-04 07:50:00	2
5	3	3	2026-05-02 14:00:00	1
6	3	8	2026-05-04 13:30:00	2
7	4	4	2026-05-02 18:45:00	1
8	4	9	2026-05-05 19:00:00	2
9	5	5	2026-05-03 10:20:00	1
10	5	10	2026-05-06 11:15:00	2

37. ROW_NUMBER(): Нумерує поїздки (1, 2, 3...) хронологічно для кожного окремого клієнта.

```
1  -- Запит 38. RANK(): Скласти рейтинг водіїв за кількістю виконаних поїздок
2  SELECT driver_id, COUNT(ride_id) AS rides_done,
3          RANK() OVER(ORDER BY COUNT(ride_id) DESC) AS driver_rank
4  FROM rides
5  WHERE status = 'completed'
6  GROUP BY driver_id;
```

	driver_id integer	rides_done bigint	driver_rank bigint
1	1	4	1
2	3	3	2
3	2	2	3

38. RANK(): Складає рейтинг водіїв на основі кількості виконаних поїздок (при однаковій кількості ранг пропускається).

```
1  -- Запит 39. DENSE_RANK(): Рейтинг поїздок за кілометражем (без пропусків місць)
2  SELECT ride_id, distance_km,
3          DENSE_RANK() OVER(ORDER BY distance_km DESC) AS distance_rank
4  FROM rides;
```

	ride_id [PK] integer	distance_km numeric (5,2)	distance_rank bigint
1	4	22.40	1
2	9	18.20	2
3	6	15.00	3
4	1	12.50	4
5	7	9.50	5
6	3	8.00	6
7	10	7.70	7
8	2	5.20	8
9	8	4.00	9
10	5	3.10	10

39. DENSE_RANK(): Створює щільний рейтинг поїздки за дистанцією (найдовша = 1, наступна = 2, без пропусків місць).

```
1  -- Запит 40. LAG(): Аналіз трендів - порівняння ціни поточної поїздки з попередньою за часом
2  SELECT ride_id, ride_date, price,
3         LAG(price) OVER(ORDER BY ride_date) AS previous Ride Price,
4         price - LAG(price) OVER(ORDER BY ride_date) AS price_difference
5  FROM rides;
```

	ride_id [PK] integer	ride_date timestamp without time zone	price numeric (7,2)	previous Ride Price numeric	price_difference numeric
1	1	2026-05-01 08:30:00	250.00	[null]	[null]
2	2	2026-05-01 09:15:00	120.00	250.00	-130.00
3	3	2026-05-02 14:00:00	160.00	120.00	40.00
4	4	2026-05-02 18:45:00	450.00	160.00	290.00
5	5	2026-05-03 10:20:00	85.00	450.00	-365.00
6	6	2026-05-03 22:10:00	300.00	85.00	215.00
7	7	2026-05-04 07:50:00	190.00	300.00	-110.00
8	8	2026-05-04 13:30:00	100.00	190.00	-90.00
9	9	2026-05-05 19:00:00	380.00	100.00	280.00
10	10	2026-05-06 11:15:00	150.00	380.00	-230.00

40. LAG(): Виконує аналіз трендів — порівнює вартість поточної поїздки з попередньою за часом поїздки і показує різницю.

Висновок

У ході виконання лабораторної роботи №3 я закріпив теоретичні знання та здобув практичні навички у складанні складних SQL-запитів до розробленої мною реляційної бази даних «Служба таксі». Було успішно написано та протестовано 40 SQL-запитів, які покривають усі вимоги завдання.

Я реалізував базову фільтрацію (логічні оператори, BETWEEN, IN), групування та агрегацію даних (COUNT, SUM, AVG). Було на практиці перевірено роботу всіх типів з'єднань (JOIN), що дозволило ефективно комбінувати дані з таблиць клієнтів, водіїв та поїздки. Окрім цього, я побудував складні аналітичні запити з використанням підзапитів різного рівня вкладеності (EXISTS, IN), операцій над множинами (UNION, INTERSECT, EXCEPT), а також структурував код за допомогою CTE. Особливу увагу було приділено використанню віконних функцій

(**ROW_NUMBER**, **RANK**, **LAG**), що дозволило виконати глибоку аналітику даних без зміни їхньої структури.